

ISO SECURE

Guía de Estudio: Análisis de Seguridad

Pruebas dinámicas (DAST) y Configuración Segura

Material didáctico — técnico pero accesible

Agustín Peralta — Universidad de San Buenaventura

Mayo de 2026

Resumen

Esta guía profundiza en la **Etapla 4 del proyecto ISO SECURE**: el análisis de seguridad dinámico (DAST) y la configuración segura del sistema en producción. Está pensada para que cualquier persona con conocimientos básicos entienda *cómo se le hacen pruebas de seguridad a una aplicación web ya construida* y *cómo se endurece la infraestructura* para reducir el área de ataque. Las primeras etapas (requerimientos, diseño, codificación) aparecen sólo como contexto. El foco está en **cómo atacar el propio sistema antes que lo haga un externo** y en **cómo configurar cada capa para que aguante**.

Índice

1. Contexto en una página	3
2. ¿Por qué hacemos análisis dinámico?	3
3. Las herramientas que usamos (y para qué sirve cada una)	4
4. Estrategia: el ataque en 4 capas	4
4.1. Capa 1 — Reconocimiento (sin login)	4
4.2. Capa 2 — Autenticación	5
4.3. Capa 3 — Autorizado (con cuenta de prueba)	5
4.4. Capa 4 — Fuzzing	6
5. Catálogo de hallazgos esperados (con su mitigación)	6
6. Configuración Segura — el "hardening"	7
6.1. Sistema operativo del host	7
6.2. Docker y contenedores	7
6.3. Nginx (reverse proxy + TLS)	8
6.4. PostgreSQL (Supabase)	8

6.5. Aplicación FastAPI	9
6.6. Frontend React	9
7. Gestión de secretos en producción	9
8. Despliegue seguro: blue-green con rollback	9
9. Respuesta a Incidentes (cuando algo igual sale mal)	10
10. Glosario rápido	10
11. Cómo ver todo esto en el sistema desplegado	10
12. Cumplimiento ISO 27001:2022 (la parte que importa para la certificación)	10
13. Cierre	10

1. Contexto en una página

ISO SECURE es una aplicación web para gestión del SGSI bajo ISO/IEC 27001:2022, ya construida. Stack: **FastAPI (Python) + React + PostgreSQL Supabase**, desplegada en **Coolify** en <https://iso-secure.agustinynatalia.site/>.

Antes de llegar a esta guía hicimos:

- **Etapas 1 —** Requerimientos (qué tenía que hacer).
- **Etapas 2 —** Arquitectura + modelado de amenazas STRIDE/PASTA (24 amenazas catalogadas, 15 riesgos).
- **Etapas 3 —** Codificación con SAST (análisis estático del código: ruff, bandit, semgrep).

Ahora, en esta guía, el foco es la **Etapas 4**: probar la app corriendo *como si fuéramos atacantes* (DAST) y endurecer la configuración de toda la pila para que aguante esas pruebas y las del mundo real.

Diferencia clave: SAST vs. DAST

SAST (Static Application Security Testing): analizar el código fuente *sin* correr el programa. Es como leer una receta para detectar errores.

DAST (Dynamic Application Security Testing): *ejecutar* el programa y atacarlo con peticiones reales. Es como cocinar la receta y probarla.

Ambos son obligatorios. Detectan cosas distintas. SAST encuentra problemas en el código (ej. SQL concatenado); DAST encuentra problemas de configuración y comportamiento que sólo aparecen en runtime (ej. una cabecera HTTP que no estaba puesta, una sesión que se reutiliza).

2. ¿Por qué hacemos análisis dinámico?

Imaginemos un edificio recién construido. El SAST sería revisar los planos para ver si las paredes tienen el grosor reglamentario. El DAST sería entrar al edificio con un martillo y golpear paredes a ver qué pasa.

Las paredes pueden estar bien dibujadas en el plano y haberse construido mal. O peor: la puerta principal puede estar bien pero alguien dejó la ventana del baño abierta. **Sólo atacando el edificio terminado descubrimos esas cosas.**

Cosas que SAST no puede ver y DAST sí:

- Cabeceras HTTP que faltan en producción aunque estén configuradas en código (porque Nginx las elimina).
- Endpoints que olvidaste poner detrás de autenticación.
- Cookies sin atributo `Secure` o `HttpOnly`.
- Cifrados débiles activos en el servidor TLS.
- Versiones expuestas del servidor (ej. `Server: nginx/1.18`).
- Mensajes de error con stack traces en producción.

- Configuraciones incorrectas que no están en el código de la app sino en la infraestructura.
- Bugs de lógica que sólo se reproducen con secuencias específicas de peticiones.

3. Las herramientas que usamos (y para qué sirve cada una)

Herramienta	Qué hace	Tipo
OWASP ZAP	Proxy interceptor + spider que recorre la app + scan activo con cientos de payloads. Es el "todo en uno" del DAST.	General
Nuclei	Ejecuta plantillas YAML mantenidas por la comunidad. Cada plantilla busca una vulnerabilidad o configuración específica (CVE, panel expuesto, default credential).	Plantillas
sqlmap	Especializado en inyección SQL. Detecta y explota automáticamente todos los tipos: error-based, blind, time-based.	Especializado
nikto	Escáner clásico de servidor web. Busca archivos sensibles (<code>.git</code> , <code>backup.sql</code>), cabeceras inseguras y versiones.	Servidor
testssl.sh	Auditoría completa de la configuración TLS/SSL: protocolos, suites de cifrado, certificados, vulnerabilidades conocidas (HEART-BLEED, ROBOT, POODLE).	TLS
ffuf	"Fuzzer" web ultrarápido. Sirve para descubrir endpoints y parámetros ocultos disparando miles de peticiones contra un diccionario.	Fuzzing
JWT_Tool	Toolkit específico para tokens JWT: detecta firma débil, <code>alg=none</code> , kid traversal, falsificación.	API/Auth
Burp Suite Community	Equivalente a ZAP. Más usado en pentesting manual; tiene proxy interceptor y repeater.	Manual
Trivy	Escanea imágenes Docker en busca de CVEs en paquetes del sistema operativo de la imagen y dependencias.	Contenedores

4. Estrategia: el ataque en 4 capas

No se ataca todo a la vez. Se va por capas, cada una asumiendo más conocimiento del sistema:

4.1. Capa 1 — Reconocimiento (sin login)

Es lo que un atacante anónimo en internet puede ver. Sólo navegando la URL pública.

- **ZAP Spider:** recorre el sitio automáticamente y mapea todas las URLs visibles.
- **nikto:** busca archivos comunes sensibles. ¿Hay un `/.git/`? ¿Un `/backup.sql`? ¿Un `/.env`?
- **testssl.sh:** validar que sólo TLS 1.2+ esté activo y los cifrados sean modernos.
- **Nuclei templates de exposures:** comprobar paneles administrativos expuestos, repos de código, archivos de configuración.

Comando real que ejecutamos

```
nuclei -u https://iso-secure.agustinynatalia.site -t exposures/ -t misconfiguration/ -severity critical,high -o reporte-nuclei.txt
```

4.2. Capa 2 — Autenticación

Ahora atacamos el login.

- **Enumeración de usuarios:** ¿el mensaje "usuario no existe" es distinto a "contraseña incorrecta"? Si lo es, un atacante puede sacar la lista de usuarios.
- **Brute force:** ¿hay throttle o lockout después de N intentos?
- **Credential stuffing:** probar credenciales filtradas en otras brechas (Have I Been Pwned).
- **Pruebas de JWT:** validar que el sistema rechaza tokens con `alg=none`, firmas modificadas, `exp` vencido o futuras.

4.3. Capa 3 — Autorizado (con cuenta de prueba)

Con una cuenta válida (rol *supervisor* o *analista*, con bajos privilegios), atacamos la lógica.

- **IDOR (Insecure Direct Object Reference):** cambiar el ID en una URL. ¿Si abro `/incidents/123` y cambio a `/incidents/124` veo datos ajenos?
- **Cross-tenant:** con un token de la empresa A, ¿puedo ver datos de la empresa B?
- **Privilege escalation:** con rol supervisor, ¿puedo ejecutar acciones de admin?
- **Tamper de parámetros:** mandar payloads modificados al body de la petición.

Tres riesgos críticos para multi-tenant

1. **Bypass de filtro por empresa:** si el código se confía de un `empresa_id` en el body de la petición, el atacante lo cambia y ve datos de otra empresa.
2. **Predicción de IDs:** si los IDs son secuenciales (1, 2, 3...), un atacante puede iterar.
3. **Endpoints olvidados sin `require_role()`:** basta uno sin el guard para abrir el sistema.

Mitigaciones aplicadas en ISO SECURE: el `empresa_id` se infiere del JWT, nunca del request; los IDs son UUID; cada endpoint tiene `require_role()` declarado.

4.4. Capa 4 — Fuzzing

Disparamos peticiones masivas con datos absurdos:

- Strings de 1MB en campos pensados para 100 caracteres.
- Enteros negativos donde se esperan positivos.
- Fechas de año 9999 o año -1.
- Inyecciones: SQL, NoSQL, comandos del sistema, plantilla (SSTI), LDAP, XPath.
- Caracteres Unicode, emojis, bytes nulos.

5. Catálogo de hallazgos esperados (con su mitigación)

Esta tabla sintetiza qué vulnerabilidades buscamos, qué severidad tienen y cómo las mitigamos:

ID	Hallazgo	Sev.	Mitigación aplicada
D-01	Cookies sin <code>HttpOnly/Secure</code>	Alta	Atributos <code>HttpOnly; Secure; SameSite=Lax</code>
D-02	CORS demasiado permisivo (*)	Alta	Whitelist explícita por entorno (sólo dominio propio)
D-03	Cabeceras de seguridad faltantes	Media	Nginx con CSP, HSTS, X-Frame-Options, X-Content-Type-Options
D-04	Stack traces en producción	Media	<code>DEBUG=False</code> + handler genérico
D-05	<code>Robots.txt</code> expone rutas internas	Baja	Quitar referencias internas
D-06	<code>Server: nginx/1.x</code> revela versión	Baja	<code>server_tokens off;</code>
D-07	TLS 1.0 / 1.1 habilitado	Alta	Solo TLS 1.2+ (ideal 1.3)
D-08	Suites RC4 / 3DES activas	Alta	Suite Mozilla intermediate
D-09	Brute force sin <code>lockout</code>	Alta	Rate limit Nginx + <code>lockout</code> Supabase
D-10	JWT <code>alg=none</code> aceptado	Crítica	Validación contra Supabase <code>/auth/v1/user</code>
D-11	IDOR en <code>/incidents/{id}</code>	Crítica	Verificación de ownership en service layer
D-12	Bypass cross-tenant via parámetro	Crítica	Helper <code>_empresa_filter</code> infiere del JWT
D-13	SSRF en webhooks n8n	Alta	Allowlist de hosts salientes

ID	Hallazgo	Sev.	Mitigación aplicada
D-14	Open redirect en <code>/login?return_to</code>	Media	Validar dominio destino
D-15	Falta de rate limit global	Alta	slowapi + Nginx <code>limit_req</code>

6. Configuración Segura — el "hardening"

Análisis dinámico encuentra el problema; el hardening lo previene. Endurecimos capa por capa.

6.1. Sistema operativo del host

- Distribución mínima (Debian slim o Alpine): menos software = menos vulnerabilidades.
- Actualizaciones automáticas de seguridad (`unattended-upgrades`).
- SSH endurecido: sólo claves, no contraseñas, puerto no estándar, `fail2ban`.
- Firewall `ufw`: sólo 22, 80 y 443 abiertos. Todo lo demás bloqueado.
- Sin servicios innecesarios; auditoría con `systemd-analyze`.

6.2. Docker y contenedores

- Imágenes base **oficiales y versionadas** (no usar `latest`, que cambia bajo los pies).
- Multi-stage builds: la imagen final no contiene compiladores ni código fuente, sólo lo necesario para correr.
- Usuario **no-root** dentro del contenedor: si te lo comprometen, no tiene privilegios.
- `-cap-drop=ALL`: quitar todas las "capabilities" de Linux y devolver sólo las indispensables.
- Read-only filesystem cuando se puede.
- Escaneo con **Trivy** antes de promover una imagen a producción.

Dockerfile endurecido (extracto)

```
FROM python:3.13-slim AS builder
RUN pip install --user -r requirements.txt

FROM python:3.13-slim
RUN groupadd -r app && useradd -r -g app -u 10001 app
COPY --from=builder /root/.local /home/app/.local
COPY --chown=app:app . /app
USER app
HEALTHCHECK --interval=30s CMD python -c "...
CMD ["uvicorn", "main:app", "--host", "0.0.0.0"]
```

6.3. Nginx (reverse proxy + TLS)

Aquí se ganan o pierden la mayoría de los puntos en una auditoría DAST. Lo que aplicamos:

- **TLS 1.2 mínimo, TLS 1.3 preferido.**
- Suite de cifrados según **Mozilla SSL Configuration Generator** (perfil intermediate).
- `server_tokens off`; para no revelar la versión de Nginx.
- Rate limiting por IP en endpoints sensibles (login).
- Buffer sizes ajustados para mitigar ataques tipo Slowloris.

Cabeceras HTTP de seguridad obligatorias:

```
1 add_header Strict-Transport-Security
2     "max-age=31536000; includeSubDomains; preload" always;
3 add_header X-Content-Type-Options "nosniff" always;
4 add_header X-Frame-Options "DENY" always;
5 add_header Referrer-Policy "strict-origin-when-cross-origin"
6     always;
7 add_header Permissions-Policy
8     "geolocation=(), microphone=(), camera=()" always;
9 add_header Content-Security-Policy
10    "default-src 'self';
11    script-src 'self' 'unsafe-inline';
12    connect-src 'self' https://*.supabase.co;
13    frame-ancestors 'none';" always;
```

6.4. PostgreSQL (Supabase)

- Conexiones solo con `sslmode=require`.
- Roles separados: `app_user` (sólo CRUD) y `migrator` (sólo DDL para migraciones).
- **Sin SUPERUSER** en la cadena de conexión de la aplicación.
- **Row Level Security (RLS)** en tablas multi-tenant: la propia DB rechaza la consulta si el filtro no corresponde.
- Backups automáticos diarios + **Point-in-Time Recovery 7 días** (puedes restaurar a cualquier segundo de la última semana).
- Encriptación en reposo activa por defecto en Supabase.

6.5. Aplicación FastAPI

- `DEBUG=False` en producción.
- `docs_url=None` (o detrás de auth admin) para no exponer la documentación OpenAPI.
- Trusted Hosts middleware con whitelist de dominios.
- Middleware de logging estructurado con `request_id`.
- GZip middleware con `minimum_size` para evitar el ataque BREACH.

6.6. Frontend React

- Source maps desactivados en producción (no se entrega código original al navegador).
- Subresource Integrity (SRI) en scripts externos.
- Sólo variables `VITE_*` (públicas por diseño); ningún secreto en el bundle.

7. Gestión de secretos en producción

Un secreto en el código fuente es un boleto para la brecha. Las reglas que aplicamos:

1. **Nunca en el código.** Detección automática con `detect-secrets` en pre-commit y CI.
2. **Nunca en logs.** Filtros que enmascaran campos sensibles.
3. **Rotación periódica:** 90 días para credenciales operativas.
4. **Mínimo privilegio:** cada servicio con su propio set de credenciales.
5. **Cifrado en reposo:** vault del proveedor (Supabase Vault, GitHub Secrets, Coolify Secrets).

8. Despliegue seguro: blue-green con rollback

Toda la estrategia anterior se rompe si un mal release tira el sistema. Por eso usamos despliegue **blue-green**:

1. Coolify levanta una versión nueva del contenedor (*green*) en paralelo a la actual (*blue*).
2. Healthcheck `/health` debe pasar antes de aceptar tráfico.
3. Si pasa, se hace swap del tráfico desde *blue* a *green*.
4. Si falla, se descarta *green* y *blue* sigue sirviendo. **Cero downtime, cero riesgo.**

Nivel	Descripción	Tiempo respuesta
P0 (Crítico)	Brecha activa, indisponibilidad total, fuga de datos	15 min
P1 (Alto)	Vulnerabilidad explotable confirmada, degradación severa	1 h
P2 (Medio)	Vulnerabilidad sin exploit público, función no crítica caída	4 h
P3 (Bajo)	Hardening pendiente, hallazgo informativo	5 días hábiles

9. Respuesta a Incidentes (cuando algo igual sale mal)

Por bien que esté todo, hay que asumir que un día algo falla. Por eso tenemos plan de respuesta a incidentes (IR) con **niveles de severidad y SLA por nivel**:

Procedimiento (6 pasos, alineado a A.5.24-A.5.28 de ISO 27001:2022):

1. **Detección** — alerta automática o reporte manual.
2. **Triage** — asignar severidad y propietario.
3. **Contención** — aislar (revocar tokens, bloquear IP, deshabilitar usuario, rollback blue-green).
4. **Erradicación** — corregir causa raíz, parchear, actualizar.
5. **Recuperación** — restaurar servicio + validar integridad.
6. **Lecciones aprendidas** — post-mortem en 48 hrs + actualización del catálogo de amenazas.

10. Glosario rápido

11. Cómo ver todo esto en el sistema desplegado

El producto está en línea y se puede consultar la documentación técnica sin login:

Repositorio con el código y los PDFs de las 5 etapas: https://github.com/Agustin1231/iso_secure.

12. Cumplimiento ISO 27001:2022 (la parte que importa para la certificación)

El trabajo de la Etapa 4 cubre técnicamente los siguientes controles del Anexo A:

13. Cierre

La idea fuerza: la seguridad de una aplicación web no se decide cuando se elige el framework. Se decide en runtime, en la infraestructura y en los procesos. Por eso esta etapa (DAST + Configuración Segura) suele ser la que más vulnerabilidades reales detecta.

La cadena que aplicamos en ISO SECURE:

Término	Significado
DAST	Dynamic Application Security Testing: atacar la app corriendo.
SAST	Static Application Security Testing: leer el código fuente.
SCA	Software Composition Analysis: revisar dependencias de terceros.
CVE	Common Vulnerabilities and Exposures: identificador único de cada vulnerabilidad conocida.
CVSS	Common Vulnerability Scoring System: puntaje 0-10 que mide gravedad de un CVE.
WAF	Web Application Firewall: filtra tráfico HTTP malicioso antes de llegar a la app.
TLS	Transport Layer Security: el "candadito" del HTTPS.
HSTS	HTTP Strict Transport Security: cabecera que obliga al navegador a usar HTTPS siempre.
CSP	Content Security Policy: cabecera que limita de qué orígenes carga el navegador.
CORS	Cross-Origin Resource Sharing: política que permite o niega peticiones entre dominios.
IDOR	Insecure Direct Object Reference: ver datos ajenos cambiando un ID en la URL.
SSRF	Server-Side Request Forgery: hacer que el servidor haga peticiones a destinos no autorizados.
JWT	JSON Web Token: token firmado que prueba quién es el usuario.
PITR	Point-in-Time Recovery: restaurar la DB a cualquier momento del pasado reciente.
IR	Incident Response: respuesta a incidentes.
SLA	Service Level Agreement: compromiso de tiempos.

1. Atacamos la propia aplicación con herramientas profesionales antes que un externo.
2. Endurecimos cada capa siguiendo CIS Benchmarks y guías oficiales.
3. Automatizamos el escaneo en cada release.
4. Diseñamos el despliegue para tolerar fallos (blue-green + rollback).
5. Documentamos cómo responder cuando algo falle (procedimiento IR + SLAs).

La regla de oro: si no probaste el sistema en producción atacándolo tú mismo, no sabés qué tan seguro es. Lo demás es opinión.

Referencias

- OWASP Web Security Testing Guide (WSTG) v4.2.
- OWASP Application Security Verification Standard (ASVS) 4.0.3.
- OWASP Top 10:2021.
- CIS Docker Benchmark v1.6 y CIS Nginx Benchmark v2.0.
- Mozilla SSL Configuration Generator.
- NIST SP 800-61 Rev. 2 — Computer Security Incident Handling Guide.
- ISO/IEC 27001:2022 — Anexo A.5 y A.8.

URL pública	Qué contiene
/	App principal (requiere login).
/threat-modeling.html	Modelado STRIDE interactivo: 24 amenazas, árboles de ataque, matriz de riesgo, mitigaciones priorizadas.
/metodologia-amenazas.html	PASTA + STRIDE integrado: comparativa de 6 metodologías, 7 etapas PASTA aplicadas, 12 vulnerabilidades mapeadas a OWASP Top 10, 15 mitigaciones priorizadas.
/arquitectura.html	Arquitectura completa: capas, despliegue, autenticación/RBAC, modelo de datos.
/docs.html	Documentación de la API: 39 endpoints en 9 routers.
/manual.html	Manual de usuario con videos por módulo.

Control	Evidencia en el proyecto
A.5.24–A.5.28 IR	Procedimiento documentado con SLAs por severidad
A.5.30 Continuidad TIC	PITR 7 días + RPO 1h / RTO 2h
A.8.7 Antimalware	Trivy en CI sobre imágenes Docker
A.8.8 Gestión vulnerabilidades técnicas	DAST con ZAP + Nuclei recurrente
A.8.9 Gestión configuración	Coolify + IaC + secretos en vault
A.8.13 Backups	Backup diario + PITR
A.8.15 Logging	JSON estructurado + retención 90d
A.8.16 Monitoreo	Prometheus + alertas en latencia y error rate
A.8.20 Seguridad redes	TLS 1.3 + ufw + rate limiting
A.8.23 Filtrado web	CSP estricto + CORS allowlist
A.8.24 Criptografía	TLS 1.3 + JWT firmado RS256
A.8.26 Requisitos de seguridad de aplicaciones	DAST como gate de release
A.8.27 Arquitectura segura	Hardening por capa
A.8.29 Pruebas de seguridad	DAST automatizado documentado